

The hypervisor, also known as a **Virtual Machine Monitor (VMM)**, serves as the fundamental architectural layer in virtualization. It acts as an intermediary between the physical hardware and the various virtual environments running on it 1.

## 1. Hardware Layer Interaction

The hypervisor sits at the base of the virtualized stack.

- **Direct Interaction (Type 1):** In bare-metal architectures, the hypervisor runs **directly on the physical hardware** 1. It has direct control over the host's physical resources, such as the CPU, RAM, and storage controllers 1.
- **Mediated Interaction (Type 2):** In hosted architectures, the hypervisor runs on a **Host Operating System**, which in turn manages the hardware 2. This adds a layer of abstraction but makes the hypervisor easier to install as a standard application 2.

## 2. Hypervisor Layer Functions

The primary responsibility of the hypervisor is to **abstract physical hardware into virtual resources** 3. Its core functions include:

- **Virtualization Management:** Creating and managing the lifecycle of Virtual Machines (VMs), including their creation, provisioning, and decommissioning 4, 5.
- **Isolation:** Ensuring that each virtual environment is isolated from the others 3. A program running in one VM cannot access the resources or data of another VM unless explicitly permitted 6.
- **Intermediation:** Intercepting requests from Guest Operating Systems and translating them into instructions the physical hardware can understand 1, 6.

## 3. Guest Operating System Interaction

From the perspective of a **Guest OS**, the hypervisor provides a virtualized set of hardware that looks like a dedicated physical machine 3.

- **Independence:** Multiple Guest OSs (e.g., Windows and Linux) can coexist on the same host, each unaware of the other's existence 1.
- **Standard Interfaces:** The Guest OS interacts with virtual drivers provided by the hypervisor rather than the actual physical hardware drivers.

## 4. Resource Allocation Mechanism

The hypervisor partitions a single physical machine into several smaller virtual ones 3.

- **Dynamic Allocation:** It monitors the resource demands of each VM and allocates physical CPU cycles, memory blocks, and storage space accordingly 1.
- **Partitioning:** Each VM is assigned a specific portion of the host's total resources. For example, if Application A requires 30% of a machine's resources, the hypervisor ensures it stays within that boundary 3, 7.

## 5. Virtual CPU and Memory Management

- **Virtual CPU (vCPU):** The hypervisor schedules the execution of Guest OS tasks on the physical CPU cores. High-performance virtualization requires allocating enough physical CPU resources to minimize the "performance overhead" introduced by this scheduling layer 8.
- **Memory Management:** The hypervisor manages a mapping between "Guest Physical Memory" (what the VM sees) and "Host Physical Memory." This ensures that memory used by one Guest OS is protected from others 1.
- **Virtual Networking:** The hypervisor often includes a **Virtual Switch (vSwitch)**, a software layer that logically separates network traffic for different VMs even if they share the same physical Network Interface Card (NIC) 9.

## 6. Diagram Explanation

A standard hypervisor architecture diagram follows this vertical stack:

1. **Hardware Layer (Bottom):** The physical CPU, RAM, and NIC 1.
2. **Hypervisor Layer (Middle):** The "Virtualization Layer" that manages resource distribution 6.
3. **Guest OS Layer (Top):** Multiple isolated operating systems running their own applications 1.
4. **Applications:** The specific software (e.g., App A, App B) running within each isolated Guest OS 1, 7.

## 10-Mark Exam-Ready Structured Notes

- **Introduction:** Define the Hypervisor (VMM) as the software layer that enables resource sharing and multi-OS coexistence 1.
- **Architectural Types:**
  - **Type 1 (Bare-Metal):** Runs on hardware (e.g., VMware ESXi). Best for efficiency 1.
  - **Type 2 (Hosted):** Runs on a Host OS (e.g., VirtualBox). Best for ease of use 2.
- **Core Architectural Principles:**
  - **Partitioning:** Breaking a physical server into smaller, independent virtual servers 3.
  - **Encapsulation:** Packaging an entire VM state (OS, apps, data) into a set of files for easy migration 10.
  - **Abstraction:** Providing a uniform virtual hardware interface to diverse Guest OSs 1.
- **Operational Management:**
  - Discuss the **VM Lifecycle** (Creation  $\rightarrow$  Provisioning  $\rightarrow$  Operation  $\rightarrow$  Decommissioning) 4, 5.
  - Explain **Resource Optimization:** Preventing physical servers from standing idle by running multiple VMs 2.
  - **Networking & I/O:** Describe the role of the **vSwitch** in bridging virtual machines to the physical network 9.
- **Conclusion:** Mention the trade-offs—while the hypervisor provides flexibility and cost savings, it introduces **performance overhead** and **management complexity** 8, 11.

